

other code that puts a nonstring value into the list. If the list should contain multiple types, we can set the type hint using `Union`. For example, the `numbers` list can contain integer and float values, so we set its type hint to

```
List[Union[int, float]] ❷.
```

The `typing` module has a separate *type alias* for each container type. Here's a list of the type aliases for common container types in Python:

`List` is for the `list` data type.

`Tuple` is for the `tuple` data type.

`Dict` is for the dictionary (`dict`) data type.

`Set` is for the `set` data type.

`FrozenSet` is for the `frozenset` data type.

`Sequence` is for the `list`, `tuple`, and any other sequence data type.

`Mapping` is for the dictionary (`dict`), `set`, `frozenset`, and any other mapping data type.

`ByteString` is for the `bytes`, `bytearray`, and `memoryview` types.

You'll find the full list of these types online at

<https://docs.python.org/3/library/typing.html#classes-functions-and-decorators>.

Backporting Type Hints with Comments

Backporting is the process of taking features from a new version of software and *porting* (that is, adapting and adding) them to an earlier version. Python's type hints feature is new to version 3.5. But in Python code that might be run by interpreter versions earlier than 3.5, you can still use type hints by putting the type information in comments. For variables, use an inline comment after the assignment statement. For functions and methods, write the type hint on the line following the `def` statement. Begin the comment with `type:`, followed by the data type. Here's an example of some code with type hints in the comments: